



Row vs Columnar Storage







Columnar Storage Format



**Columnar
Storage
Format**



**Columnar &
Row Storage
Differences**

Weather Analysis



Query: Find average temperature between timestamp **5** and timestamp **75**

PAGE 1			
Timestamp 1	Temperature 1	Humidity 1	Windspeed 1
Timestamp 2	Temperature 2	Humidity 2	Windspeed 2
...	...	...	...
Timestamp 10	Temperature 10	Humidity 10	Windspeed 10

Row Storage

Find average temperate between timestamp 5 and timestamp 75

PAGE 1			
Timestamp 1	Temperature 1	Humidity 1	Windspeed 1
Timestamp 2	Temperature 2	Humidity 2	Windspeed 2
...	...	...	...
Timestamp 10	Temperature 10	Humidity 10	Windspeed 10

PAGE 2			
Timestamp 11	Temperature 11	Humidity 11	Windspeed 11
...	...	...	...
Timestamp 20	Temperature 20	Humidity 20	Windspeed 20

Columnar Storage

Find average temperate between timestamp 5 and timestamp 75

PAGE 1	PAGE 2	PAGE 3	PAGE 4
Timestamp 1	Temperature 1	Humidity 1	Windspeed 1
Timestamp 2	Temperature 2	Humidity 2	Windspeed 2
...	...	...	...
Timestamp 40	Temperature 40	Humidity 40	Windspeed 40

Row vs Columnar Storage

FEATURE	ROW STORAGE	COLUMNAR STORAGE
Data Organization	All attributes of a record stored together	Each attribute stored in separate files
Ideal Use Case	Transactional workloads (OLTP), where entire records are accessed frequently.	Analytical workloads (OLAP), which often require only a subset of columns
Access Pattern	Efficient for accessing full rows	Efficient for accessing specific columns
Compression	Limited compression potential	High compression potential due to similar data types within columns

Generating Synthetic Weather Dataset

Table with four columns
and 1 million rows.

Store table as a set of
4 KB pages

```
const int NUM_COLS = 4;    // Timestamp, Temperature, Humidity, Wind Speed
const int NUM_ROWS = 1000000; // Table size
const int PAGE_SIZE = 4096; // Data stored in pages of 4096 bytes
```

Generating Synthetic Weather Dataset

Temperature, humidity, and wind speed are randomly generated.

Uses sequential timestamps to simulate real-world weather data.

```
void generateData(std::vector<std::vector<int>> &data) {  
    std::normal_distribution<> temp_change(-0.5, 5);  
    std::uniform_int_distribution<> humidity_dis(0, 100);  
    std::uniform_int_distribution<> wind_speed_dis(0, 100);  
    ...  
}
```


Generated Weather Dataset

Timestamp	Temperature	Humidity	Wind Speed
1609459200	40	74	12
1609459202	37	84	14
1609459207	35	6	31
1609459210	35	90	59

Row Storage

Single file storing rows (tuples) sequentially

```
std::ofstream rowFile("row_storage.dat", std::ios::binary);  
for (const auto &row : data) {  
    rowFile.write(reinterpret_cast<const char*>(row.data()), NUM_COLS * sizeof(int));  
}
```

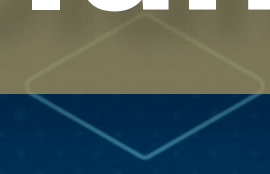
Columnar Storage

Separate files for each column in the table in binary format.

```
std::vector<std::ofstream> colFiles(NUM_COLS);  
for (int i = 0; i < NUM_COLS; ++i) {  
    colFiles[i].open("column_storage_" + std::to_string(i) + ".dat", std::ios::binary);  
}
```




Querying Data with Row & Columnar Storage Formats



Querying Data with Row Storage Format

Limitation: Processes entire rows, even though only one column (temperature) is needed.

```
double queryAverageTemperatureRowStorage(int startTimestamp, int endTimestamp) {
    std::ifstream rowFile("row_storage.dat", std::ios::binary);
    int sumTemperatures = 0, count = 0;
    while (rowFile.read(reinterpret_cast<char*>(row.data()),
                        NUM_COLS * sizeof(int))) {
        if (row[0] >= startTimestamp && row[0] <= endTimestamp) {
            sumTemperatures += row[1];
            count++;
        }
    }
    return count > 0 ? static_cast<double>(sumTemperatures) / count : 0.0;
}
```

Columnar Storage Format Query: Part 1

Only the timestamp and temperature files are accessed, making the query faster than row storage.

```
long long queryColumnarStorage(int &filterPagesRead, int &aggregatePagesRead) {  
    std::ifstream filterFile("column_storage_" + std::to_string(FILTER_COLUMN) +  
                             ".dat", std::ios::binary);  
    std::ifstream aggregateFile("column_storage_" +  
                                std::to_string(AGGREGATE_COLUMN) + ".dat",  
                                std::ios::binary);  
  
    long long sum = 0;  
    std::unordered_map<int, std::vector<int>> pageOffsetMap;  
    ...  
}
```

Columnar Storage Format Query: Part 2

Access only the timestamp and temperature files.

```
// Read the filter column and collect row offsets for qualifying rows
for (int startRow = 0; startRow < NUM_ROWS; startRow += INTS_PER_PAGE) {
    auto filterPage = readColumnPage(filterFile, startRow, filterPagesRead);
    for (size_t i = 0; i < filterPage.size(); ++i) {
        // std::cout << filterPage[i] << "\n";
        if (filterPage[i] > FILTER_THRESHOLD) {
            pageOffsetMap[startRow / INTS_PER_PAGE].push_back(i);
        }
    }
}
```


Columnar Storage Format Query: Part 3

Aggregate the temperature column using the collected tuple offset

```
// Read the aggregate column using the collected row offsets
for (const auto &[pageIndex, offsets] : pageOffsetMap) {
    auto aggregatePage = readColumnPage(aggregateFile, pageIndex * INTS_PER_PAGE,
                                         aggregatePagesRead);
    for (const auto &rowIndex : offsets) {
        sum += aggregatePage[rowIndex];
    }
}
```

Performance Comparison

Fewer pages are read from disk with columnar storage

Row Storage Query Result : 7358
Row Storage Query Time : 112.1 milliseconds

Columnar Storage Query Result : 7358
Columnar Storage Query Time : 3.6 milliseconds

Pages Read:

Total Row Storage Pages Read : 3907

Total Columnar Storage Pages Read : 962

Filter Pages Read : 489

Aggregate Pages Read : 473





Columnar Storage Format



**Columnar
Storage
Format**



**Columnar &
Row Storage
Differences**



Compressed Columnar Storage







Columnar Storage and Compression

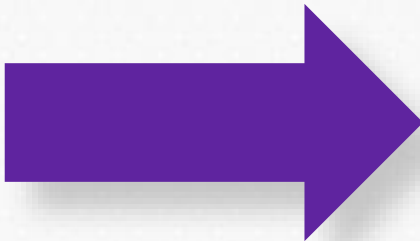


**Columnar
Storage
and
Compression**

**Compression
Algorithms**

Compression in Databases

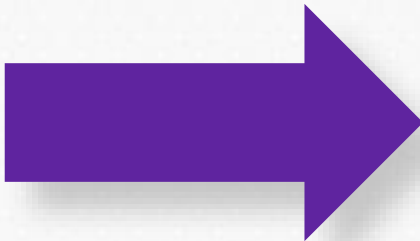
Timestamp
1609459200
1609459202
1609459207
1609459210



Timestamp
1609459200
+2
+5
+3
+2
+5
+3
+5

Compression in Databases

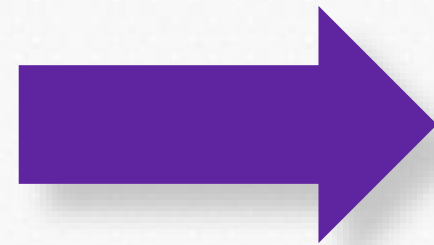
Timestamp
1609459200
1609459202
1609459207
1609459210



Timestamp
1609459200
+2
+5
+3
+2
+5
+3
+5

Compression in Databases

Timestamp
1609459200
1609459202
1609459207
1609459210



Timestamp
1609459200
+2
+5
+3
+2
+5
+3
+5

σ Delta Encoding

Compression and Columnar Storage

ROW STORAGE		
Timestamp	Product ID	Price
1609459200	AB20	50
1609459202	AB10	100
1609459207	AB10	60
1609459210	AB20	200

COLUMNAR STORAGE		
Timestamp	Product ID	Price
1609459200	AB20	50
1609459202	AB10	100
1609459207	AB10	60
1609459210	AB20	200

Compression and Columnar Storage



UNIFORM DATA
TYPES



INCREASED DATA
REDUNDANCY

Timestamp
1609459200
1609459202
1609459207
1609459210

Product ID
AB20
AB10
AB10
AB20

Price
50
100
60
200

Compression Algorithms

ALGORITHM	IDEAL USE CASE
Delta Encoding	Numeric data where values change incrementally
Bit-Packing	Numeric data with small range
Huffman Encoding	Categorical data with frequent values (e.g., strings)
Byte Dictionary Encoding	Categorical data with small range

Compression Algorithms

ALGORITHM	IDEAL USE CASE
Delta Encoding	Numeric data where values change incrementally
Bit-Packing	Numeric data with small range
Huffman Encoding	Categorical data with frequent values (e.g., strings)
Byte Dictionary Encoding	Categorical data with small range

Compression Algorithms

ALGORITHM	IDEAL USE CASE
Delta Encoding	Numeric data where values change incrementally
Bit-Packing	Numeric data with small range
Huffman Encoding	Categorical data with frequent values (e.g., strings)
Byte Dictionary Encoding	Categorical data with small range

Compression Algorithms

ALGORITHM	IDEAL USE CASE
Delta Encoding	Numeric data where values change incrementally
Bit-Packing	Numeric data with small range
Huffman Encoding	Categorical data with frequent values (e.g., strings)
Byte Dictionary Encoding	Categorical data with small range

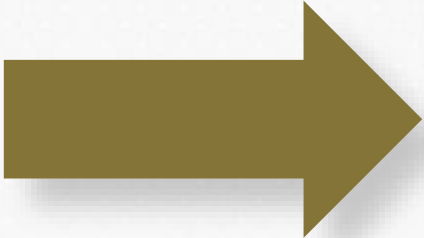
Compression Algorithms

ALGORITHM	IDEAL USE CASE
Delta Encoding	Numeric data where values change incrementally
Bit-Packing	Numeric data with small range
Huffman Encoding	Categorical data with frequent values (e.g., strings)
Byte Dictionary Encoding	Categorical data with small range

Delta Encoding

σ = CHANGE

Timestamp
1609459200
1609459202
1609459207
1609459210



Timestamp
1609459200
+2
+5
+3
+2
+5
+3
+5

Bit Packing

Compress Values with A Minimum Number of Bits

Temperature
35
40
45
50



Compressed
100011
101000
110010
110111



Packed Compressed
10001110
10001100
10110111

6-Bit Binary Code Represents Temperatures

Bit Packing: Compression

```
std::vector<uint8_t> bitPack(const std::vector<int> &values, int bitWidth) {  
    int numBits = values.size() * bitWidth;  
    int numBytes = (numBits + BITS_PER_BYTE - 1) / BITS_PER_BYTE;  
    std::vector<uint8_t> packedData(numBytes, 0);  
    int bitPos = 0;  
    for (int value : values) {  
        for (int b = 0; b < bitWidth; ++b) {  
            if (value & (1 << b)) {  
                packedData[bitPos / BITS_PER_BYTE] |= (1 << (bitPos % BITS_PER_BYTE));  
            }  
            ++bitPos;  
        }  
    }  
    return packedData;  
}
```

Bit Packing: Compression

```
std::vector<uint8_t> bitPack(const std::vector<int> &values, int bitWidth) {  
    int numBits = values.size() * bitWidth;  
    int numBytes = (numBits + BITS_PER_BYTE - 1) / BITS_PER_BYTE;  
    std::vector<uint8_t> packedData(numBytes, 0);  
    int bitPos = 0;  
    for (int value : values) {  
        for (int b = 0; b < bitWidth; ++b) {  
            if (value & (1 << b)) {  
                packedData[bitPos / BITS_PER_BYTE] |= (1 << (bitPos % BITS_PER_BYTE));  
            }  
            ++bitPos;  
        }  
    }  
    return packedData;  
}
```

Bit Packing: Decompression

```
int extractValue(const std::vector<uint8_t> &packedData, int bitWidth, int index) {  
    int value = 0;  
    int startBit = index * bitWidth;  
    for (int b = 0; b < bitWidth; ++b) {  
        if (packedData[startBit / BITS_PER_BYTE] & (1 << (startBit % BITS_PER_BYTE))) {  
            value |= (1 << b);  
        }  
        ++startBit;  
    }  
    return value;  
}
```


Bit Packing: Decompression

```
int extractValue(const std::vector<uint8_t> &packedData, int bitWidth, int index) {  
    int value = 0;  
    int startBit = index * bitWidth;  
    for (int b = 0; b < bitWidth; ++b) {  
        if (packedData[startBit / BITS_PER_BYTE] & (1 << (startBit % BITS_PER_BYTE))) {  
            value |= (1 << b);  
        }  
        ++startBit;  
    }  
    return value;  
}
```

Bit Packing: Decompression

```
int extractValue(const std::vector<uint8_t> &packedData, int bitWidth, int index) {  
    int value = 0;  
    int startBit = index * bitWidth;  
    for (int b = 0; b < bitWidth; ++b) {  
        if (packedData[startBit / BITS_PER_BYTE] & (1 << (startBit % BITS_PER_BYTE))) {  
            value |= (1 << b);  
        }  
        ++startBit;  
    }  
    return value;  
}
```

Bit Packing: Decompression

```
int extractValue(const std::vector<uint8_t> &packedData, int bitWidth, int index) {  
    int value = 0;  
    int startBit = index * bitWidth;  
    for (int b = 0; b < bitWidth; ++b) {  
        if (packedData[startBit / BITS_PER_BYTE] & (1 << (startBit % BITS_PER_BYTE))) {  
            value |= (1 << b);  
        }  
        ++startBit;  
    }  
    return value;  
}
```

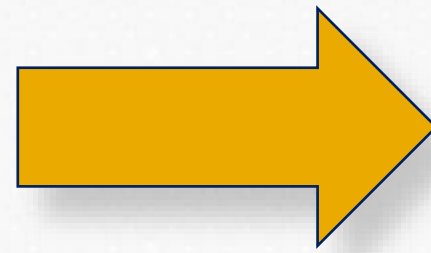



Huffman Encoding



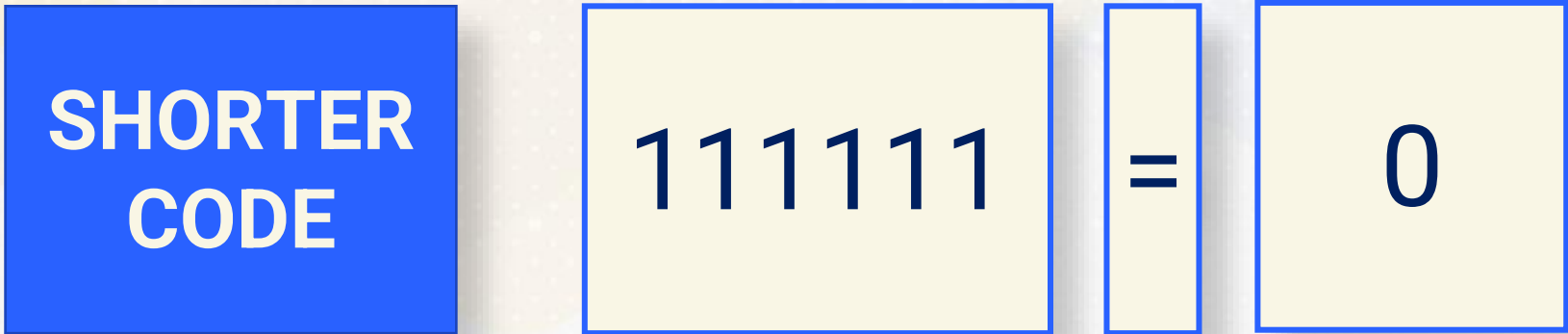
Huffman Encoding

Product ID	Frequency
111111	100
222222	60
333333	30
444444	10

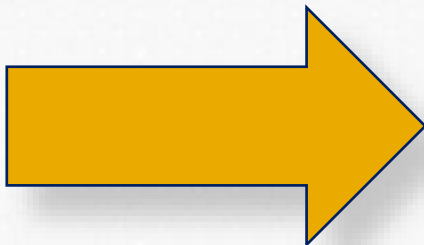


Compressed Value
0
10
110
111

Huffman Encoding



Product ID	Frequency
111111	100
222222	60
333333	30
444444	10

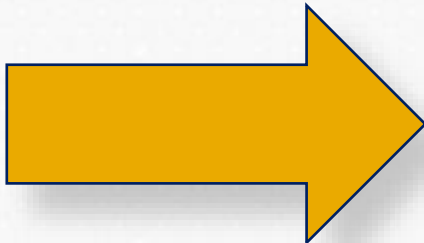


Compressed Value
0
10
110
111

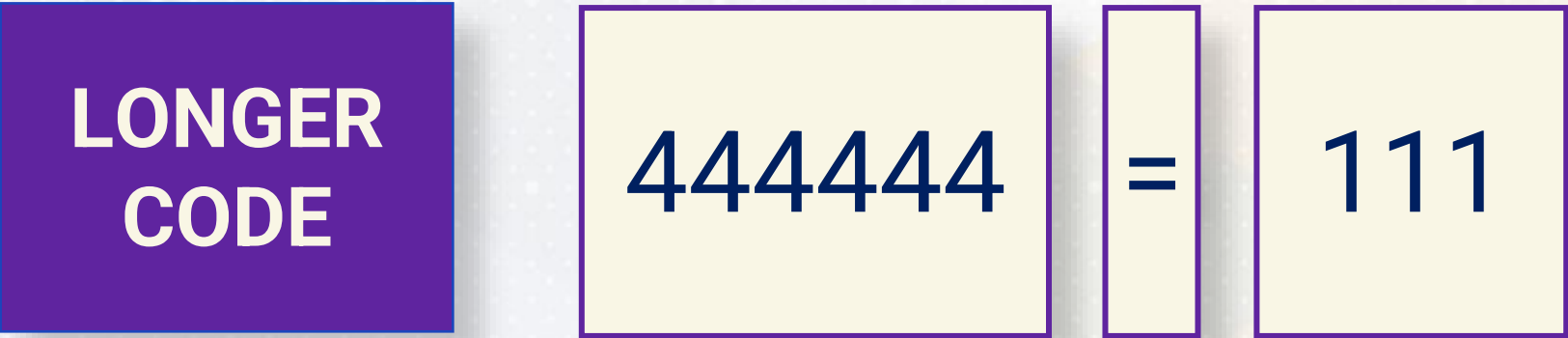
Huffman Encoding



Product ID	Frequency
111111	100
222222	60
333333	30
444444	10



Compressed Value
0
10
110
111



Huffman Encoding: Compression

```
void buildHuffmanCodes(
    const std::vector<std::pair<std::string, int>> &freq_table) {
    std::priority_queue<HuffmanNode *, std::vector<HuffmanNode *>, CompareNode> minHeap;
    for (const auto &pair : freq_table) {
        minHeap.push(new HuffmanNode(pair.first, pair.second));
    }
    while (minHeap.size() > 1) {
        HuffmanNode *left = minHeap.top(); minHeap.pop();
        HuffmanNode *right = minHeap.top(); minHeap.pop();
        HuffmanNode *newNode = new HuffmanNode("", left->freq + right->freq);
        newNode->left = left; newNode->right = right; minHeap.push(newNode);
    }
    root = minHeap.top();
    buildCodes(root, "");
}
```

Byte Dictionary Encoding

Assigns Unique Byte Values (0-255) to Each Categorical Value

Product ID
111111
222222
333333
444444



Compressed
0x01
0x02
0x03
0x04

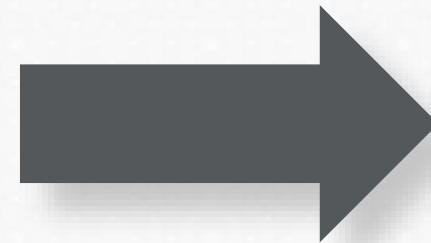
Byte Dictionary Encoding: Compression

```
for (const auto &record : table) {  
    if (dict.find(record.product_id) != dict.end()) {  
        encoded_data.push_back(dict[record.product_id]);  
    } else {  
        encoded_data.push_back(static_cast<unsigned char>(dict_size));  
        reverse_dict[static_cast<unsigned char>(dict_size)] = record.product_id;  
        dict_size++;  
    }  
}
```

Byte Dictionary Encoding: Decompression

Byte values in encoded data mapped back to Product ID

Compressed
0x01
0x02
0x03
0x04



Product ID
111111
222222
333333
444444

Illustrative Results

	QUERY TIME	QUERY RESULT
NO COMPRESSION	7.79 ms	\$2.26383e+07
HUFFMAN ENCODING	2.24 ms	\$2.26383e+07
BYTE DICTIONARY ENCODING	0.71 ms	\$2.26383e+07



Weather Dataset



Delta Encoding of Timestamp Values

Timestamps Often Increase Incrementally

```
std::vector<int> deltaTimestamps(timestamps.size());  
deltaTimestamps[0] = timestamps[0];  
std::adjacent_difference(timestamps.begin(), timestamps.end(),  
                           deltaTimestamps.begin());
```


Delta Encoding of Timestamp Values

Timestamps Often Increase Incrementally

```
std::vector<int> deltaTimestamps(timestamps.size());  
deltaTimestamps[0] = timestamps[0];  
std::adjacent_difference(timestamps.begin(), timestamps.end(),  
                           deltaTimestamps.begin());
```



deltaTimestamps

Adjustment of Temperature Values

Subtract Minimum Temperature to Reduce Range

```
int minTemp = *std::min_element(temperatures.begin(), temperatures.end());  
std::transform(temperatures.begin(), temperatures.end(),  
               adjustedTemperatures.begin(),  
               [minTemp](int temp) { return temp - minTemp; });
```

Adjustment of Temperature Values

Subtract Minimum Temperature to Reduce Range

```
int minTemp = *std::min_element(temperatures.begin(), temperatures.end());  
std::transform(temperatures.begin(), temperatures.end(),  
               adjustedTemperatures.begin(),  
               [minTemp](int temp) { return temp - minTemp; });
```


Bit-Packing of Temperature Values

Use only as many bits as necessary to store adjusted temperatures.

```
std::vector<uint8_t> bitPack(const std::vector<int> &values, int bitWidth);  
auto packedTemperatures = bitPack(adjustedTemperatures, TEMPERATURE_BIT_WIDTH);
```


Storing Compressed Data

```
std::ofstream deltaFile("delta_encoded_timestamps.dat", std::ios::binary);  
std::ofstream tempFile("packed_temperatures.dat", std::ios::binary);  
tempFile.write(reinterpret_cast<const char*>(packedTemperatures.data()),  
               packedTemperatures.size());
```

Storing Compressed Data

```
std::ofstream deltaFile("delta_encoded_timestamps.dat", std::ios::binary);  
std::ofstream tempFile("packed_temperatures.dat", std::ios::binary);  
tempFile.write(reinterpret_cast<const char*>(packedTemperatures.data()),  
               packedTemperatures.size());
```


Querying Compressed Columnar Data

```
size_t startPos = startOffset * TEMPERATURE_BIT_WIDTH;  
size_t endBitPos = (endOffset + 1) * TEMPERATURE_BIT_WIDTH;
```

Delta Decoding : [1609459200, +2, +5, ...]
-> [1609459200, 1609459202, 1609459207, ...]

Illustrative Results

	QUERY TIME	QUERY RESULT
ROW STORAGE	17.57 ms	36.9554°C
COLUMNAR STORAGE	5.13 ms	36.9554°C
COMPRESSED COLUMNAR STORAGE	4.08 ms	36.9554°C





Columnar Storage and Compression



**Columnar
Storage
and
Compression**

**Compression
Algorithms**



Vectorized Execution







Vector-at- a-Time Processing



**Vector-at-
a-Time
Processing**



**SIMD
Instructions**

Limitations of Tuple-at-a-time Processing

COLUMN VALUE	18	40	25	15
CONSTANT	> 20	> 20	> 20	> 20

Limitations of Tuple-at-a-time Processing

FUNCTION CALLS

COLUMN VALUE	18	40	25	15
CONSTANT	> 20	> 20	> 20	> 20

Limitations of Tuple-at-a-time Processing

FUNCTION CALLS

NO DATA-LEVEL PARALLELISM

COLUMN VALUE	18	40	25	15
CONSTANT	> 20	> 20	> 20	> 20

Limitations of Tuple-at-a-time Processing

FUNCTION CALLS

NO DATA-LEVEL PARALLELISM

**PIPELINE STALLS +
CACHE MISSES**

COLUMN VALUE

CONSTANT

18

> 20

40

> 20

25

> 20

15

> 20

Vector-at-a-time Processing

0	1	1	1	> 20
18	40	25	15	

Vector-at-a-time Processing

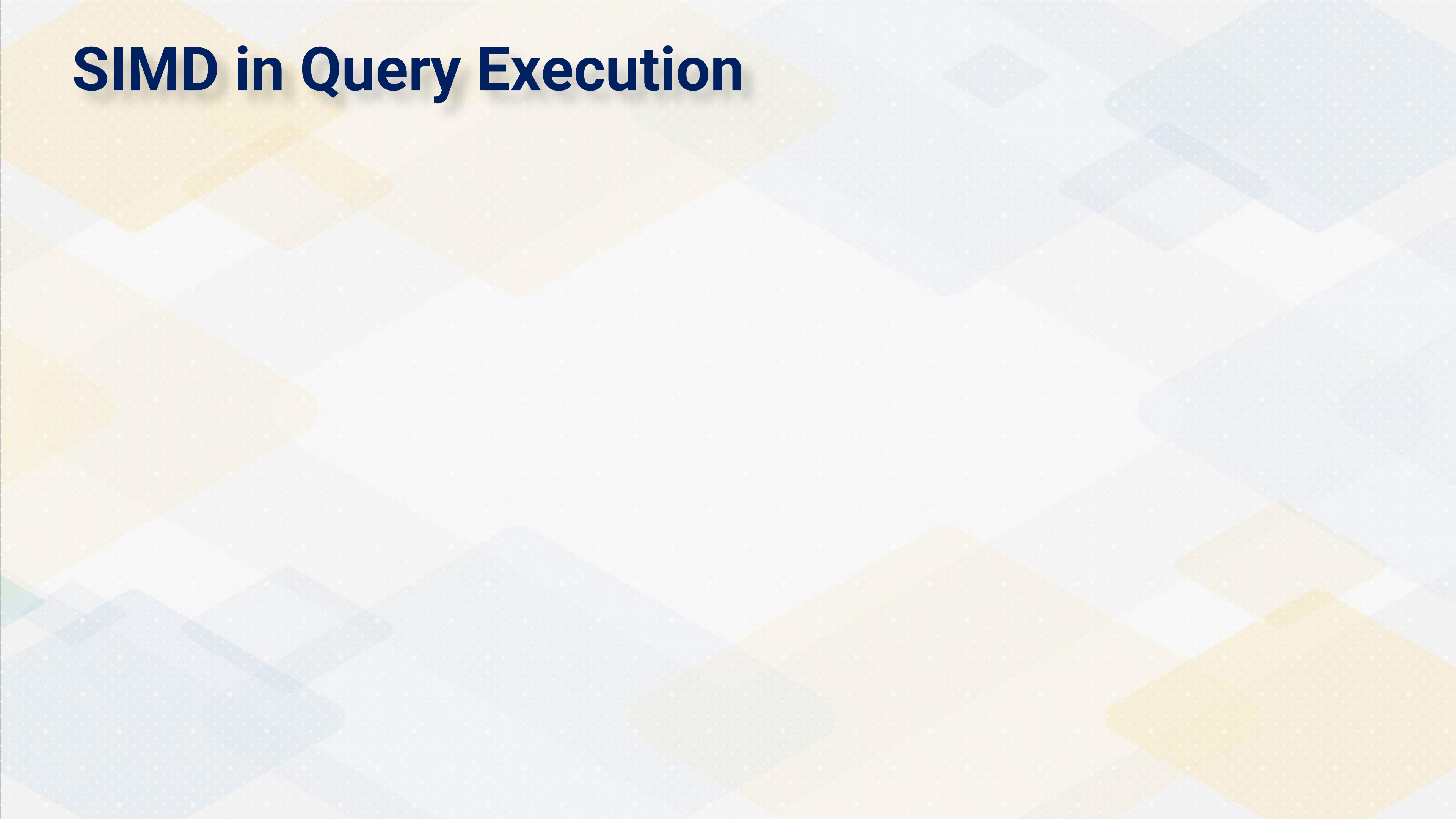
Process a vector of tuples at a time to reduce overhead of function calls

Use SIMD (Single Instruction, Multiple Data)

A single instruction operates on multiple data points simultaneously

0	1	1	1	> 20
18	40	25	15	

SIMD in Query Execution



SIMD in Query Execution

SIMD Use Cases

- Filtering
- Aggregations
- Compression

SIMD in Query Execution

SIMD Use Cases

- Filtering
- Aggregations
- Compression

SIMD Operations

- Vector loads
- Filter masks
- Horizontal reduction

Filter timestamps using SIMD

```
int32x4_t ts_vec = vld1q_s32(&data.timestamps[i]); // Load timestamps
```

```
uint32x4_t mask = vandq_u32(vcgeq_s32(ts_vec, 1),  
                             vcleq_s32(ts_vec, end)); // Mask for range
```


Filter timestamps using SIMD

```
int32x4_t ts_vec = vld1q_s32(&data.timestamps[i]); // Load timestamps
```

```
uint32x4_t mask = vandq_u32(vcgeq_s32(ts_vec, 1),  
                             vcleq_s32(ts_vec, end)); // Mask for range
```

Filter timestamps using SIMD

```
int32x4_t ts_vec = vld1q_s32(&data.timestamps[i]); // Load timestamps
```

```
uint32x4_t mask = vandq_u32(vcgeq_s32(ts_vec, 1),  
                             vcleq_s32(ts_vec, end)); // Mask for range
```


Filter timestamps using SIMD

```
int32x4_t ts_vec = vld1q_s32(&data.timestamps[i]); // Load  
timestamps
```

```
uint32x4_t mask = vandq_u32(vcgeq_s32(ts_vec, 1),  
                             vcleq_s32(ts_vec, end)); // Mask for range
```


Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Filter timestamps using SIMD

vld1q_s32	18	40	25	15	
vcgeq_s32					> 20
	0	1	1	1	
vcleq_s32					< 30
	1	0	1	1	
vandq_u32					> 20 AND < 30
	0	0	1	0	

Scalar vs SIMD Execution

**Scalar
Execution**



**SIMD
Execution**



Scalar vs SIMD Execution

**Scalar
Execution**



**One Data
Element Per
Cycle**

**Repeated
Instruction
for Each
Element**

**SIMD
Execution**



Scalar vs SIMD Execution

**Scalar
Execution**



**One Data
Element Per
Cycle**

**Repeated
Instruction
for Each
Element**

**SIMD
Execution**

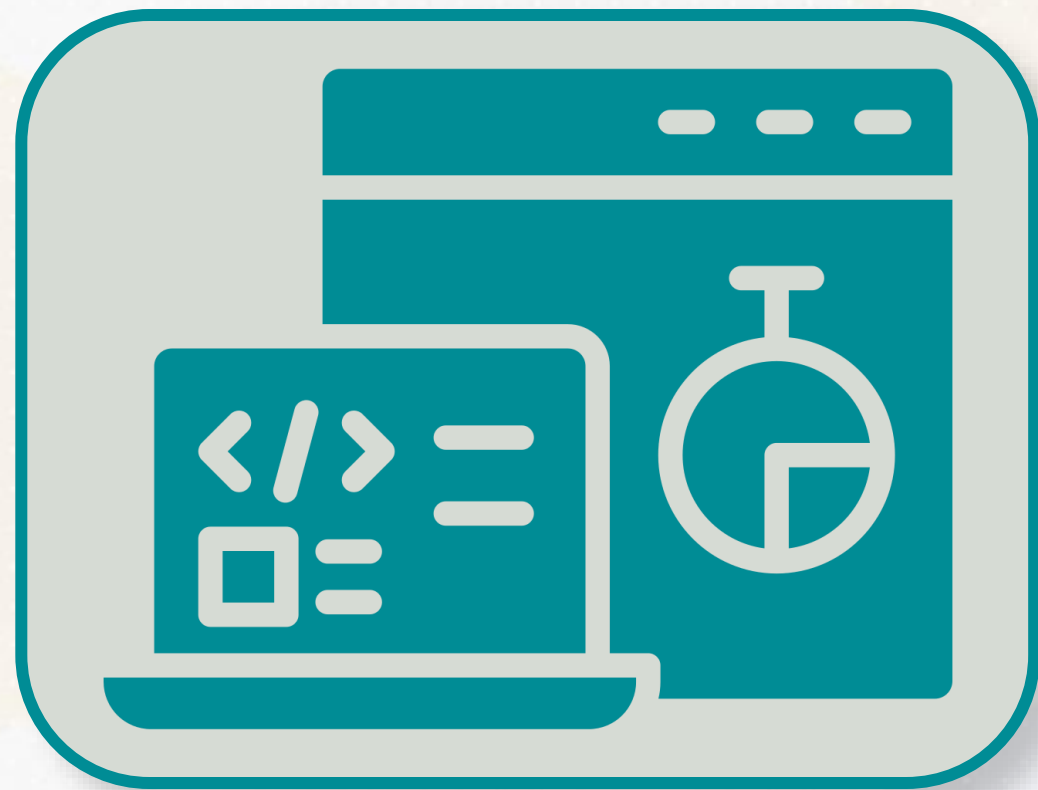


**Multiple
Data
Elements
Per Cycle**

**Batch
Execution
with Single
Instruction**

Benefits of SIMD Execution #1

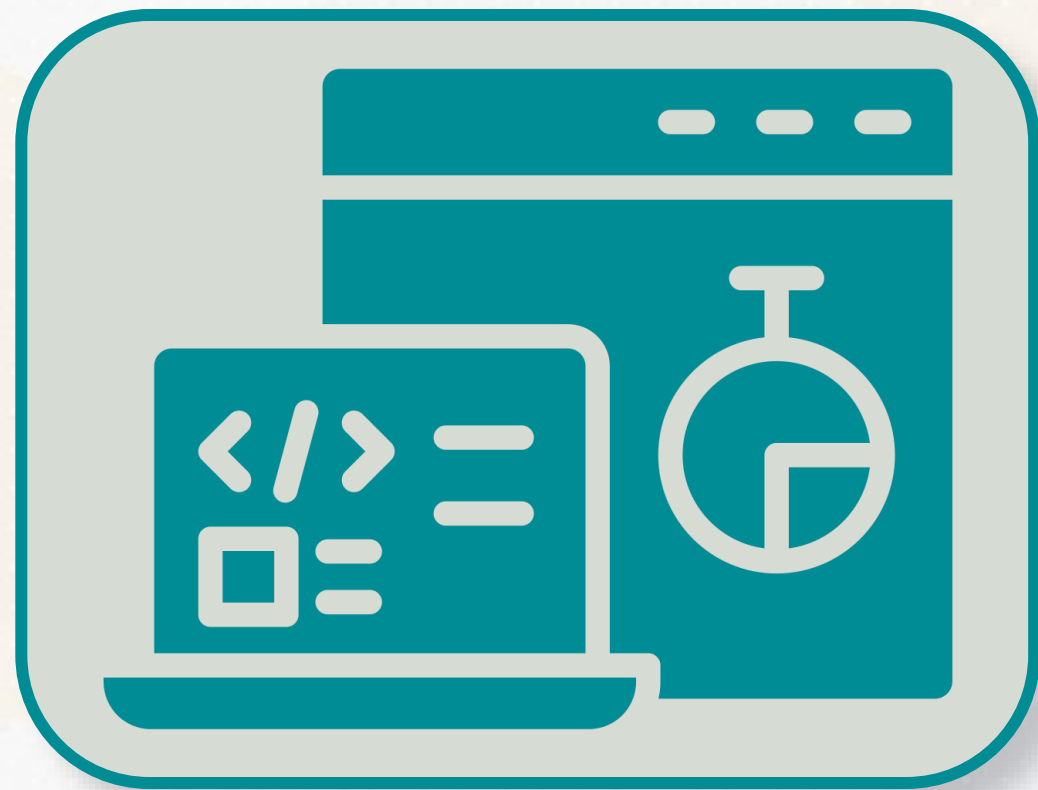
Instruction-Level Efficiency



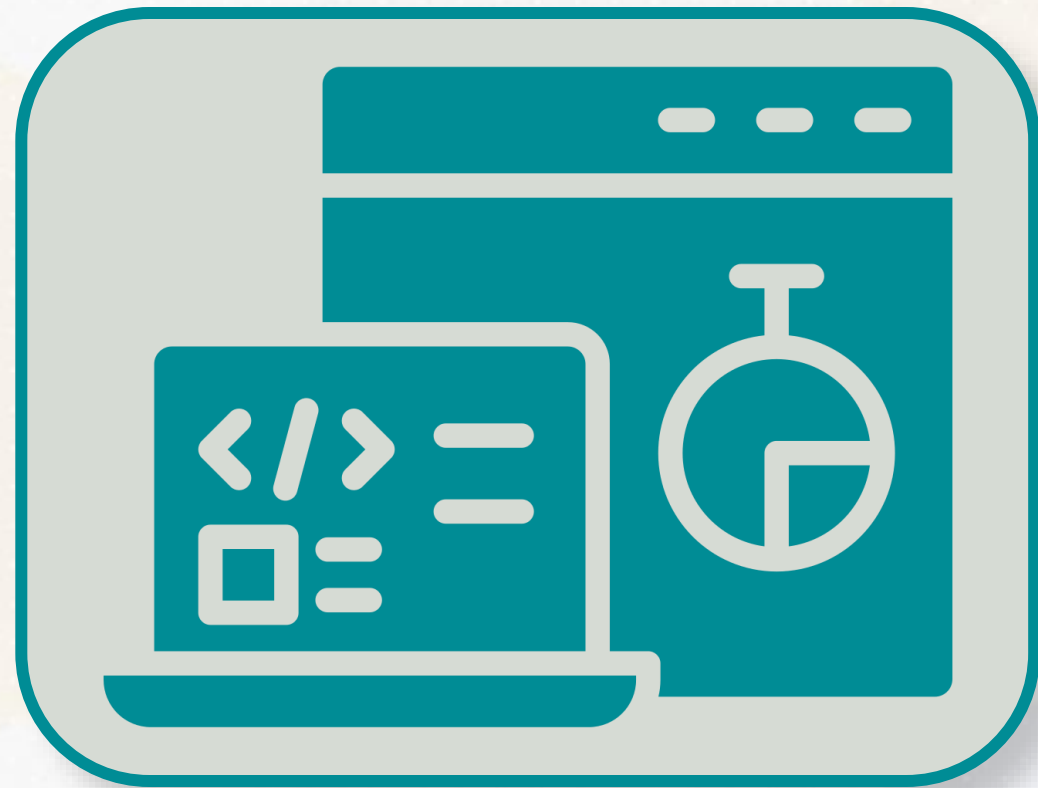
Benefits of SIMD Execution #1

Instruction-Level Efficiency

Vectorized Instructions



Benefits of SIMD Execution #1

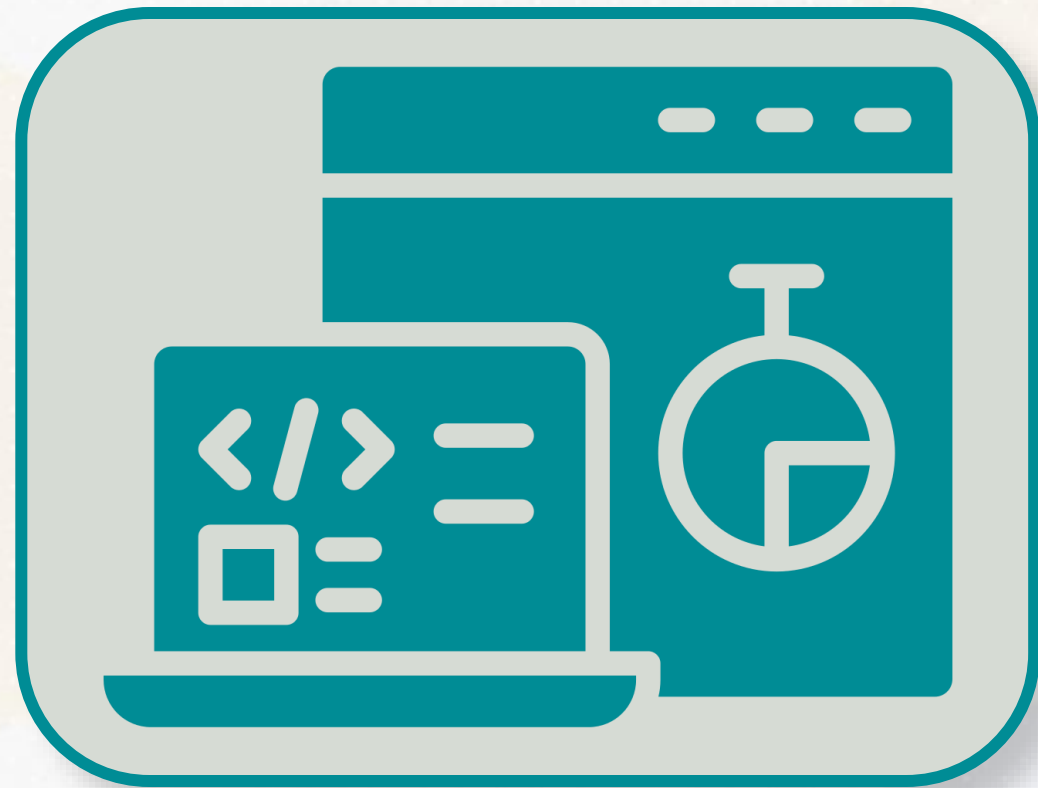


Instruction-Level Efficiency

Vectorized Instructions

N instructions for N data points

Benefits of SIMD Execution #1



Instruction-Level Efficiency

Vectorized Instructions

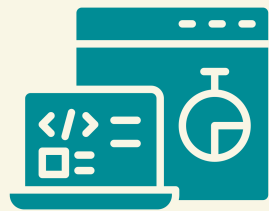
N instructions for N data points

**N/W instructions for SIMD
Vector**

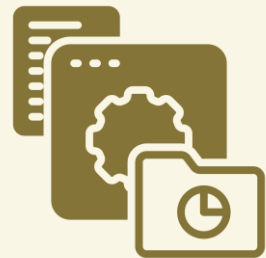
Benefits of SIMD Execution #2



**Contiguous
Memory**



SIMD Operations



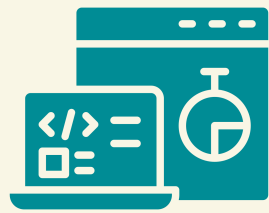
**Cache Line
Usage**

Benefits of SIMD Execution #2

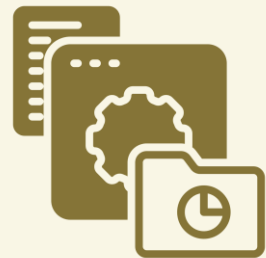


**Contiguous
Memory**

**Access Aligns with Cache
Lines**



SIMD Operations



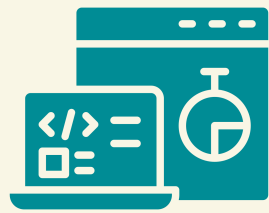
**Cache Line
Usage**

Benefits of SIMD Execution #2



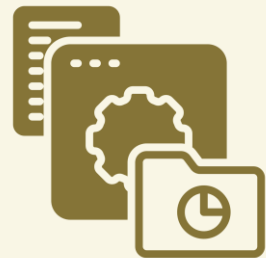
**Contiguous
Memory**

**Access Aligns with Cache
Lines**



SIMD Operations

**Columnar Layouts
Align with SIMD**



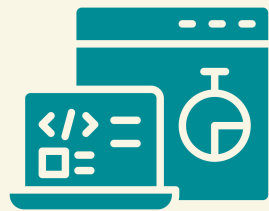
**Cache Line
Usage**

Benefits of SIMD Execution #2



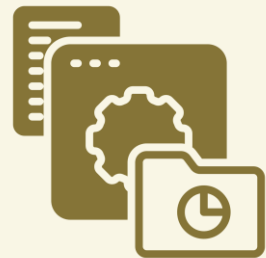
**Contiguous
Memory**

**Access Aligns with Cache
Lines**



SIMD Operations

**Columnar Layouts
Align with SIMD**



**Cache Line
Usage**

**Fully-Utilized Lines
Reduce Latency**

Benefits of SIMD Execution #3

**SIMD
minimizes
branching**

**Scalar
Execution**

**SIMD
Execution**

Benefits of SIMD Execution #3



**Operation Applied
to Vector
Elements**



Benefits of SIMD Execution #3



**Operation Applied
to Vector
Elements**



**Pipeline Stalls
with Incorrect
Branch Position**



Benefits of SIMD Execution #3



**Operation Applied
to Vector
Elements**



**Pipeline Stalls
with Incorrect
Branch Position**



**Masks Handle
Conditional
Operations & Avoid
Pipeline Stalls**

History of SIMD #1

Pixels	100	120	140	160	
					+ 20
	120	140	160	180	

History of SIMD #1

**1960s-
1970s**

**Supercomputers
and Scientific
Computing**

SIMD first used in systems
like **ILLIAC IV** for scientific
workloads

Permits simultaneous
operations on multiple
data points

Pixels	100	120	140	160	
					+ 20
	120	140	160	180	

History of SIMD #1

**1960s-
1970s**

**Supercomputers
and Scientific
Computing**

SIMD first used in systems
like **ILLIAC IV** for scientific
workloads

Permits simultaneous
operations on multiple
data points

**1980s-
1990s**

**Multimedia
Processing**

MMX (Intel, 1996): Integer
operations for multimedia

Example: Increase the
brightness of an
image's pixels by a
constant value

Pixels	100	120	140	160	
					+ 20
	120	140	160	180	

History of SIMD #1

1960s-1970s	Supercomputers and Scientific Computing	SIMD first used in systems like ILLIAC IV for scientific workloads	Permits simultaneous operations on multiple data points
1980s-1990s	Multimedia Processing	MMX (Intel, 1996): Integer operations for multimedia	Example: Increase the brightness of an image's pixels by a constant value

Pixels	100	120	140	160	
					+ 20
	120	140	160	180	

History of SIMD #2

History of SIMD #2

2000s

**Integration in
General-
Purpose CPUs**

SIMD became a
standard in
consumer CPUs

SSE/AVX (Intel):
Wide registers
for floats and
integers

NEON (ARM):
Optimized for
embedded and
mobile devices

History of SIMD #2

2000s

**Integration in
General-
Purpose CPUs**

SIMD became a
standard in
consumer CPUs

SSE/AVX (Intel):
Wide registers
for floats and
integers

NEON (ARM):
Optimized for
embedded and
mobile devices

**2010s-
Present**

**Acceleration for
Analytical
Workloads**

SIMD now powers modern
databases and big data
systems

Vectorized query
execution (e.g., Apache
Arrow)

Sensor Data Analysis

```
struct SensorData {  
    int *timestamps;    // Contiguous array of timestamps  
    float *temperatures; // Contiguous array of temperatures  
  
    SensorData(size_t count) {  
        timestamps = static_cast<int *>(aligned_alloc(16, sizeof(int) * count));  
        temperatures =  
            static_cast<float *>(aligned_alloc(16, sizeof(float) * count));  
    }  
};
```


Sensor Data Generation

```
void generateData(SensorData &data, int count) {  
    std::random_device rd;  
    std::mt19937 gen(rd());  
    std::normal_distribution<float> temp_dist(25.0, 5.0);  
    std::uniform_int_distribution<int> time_dist(1, 5);  
  
    int timestamp = START_TIMESTAMP;  
    for (int i = 0; i < count; ++i) {  
        data.timestamps[i] = timestamp;  
        data.temperatures[i] = temp_dist(gen);  
        timestamp += time_dist(gen); // Increment timestamp  
    }  
}
```


Sensor Data Generation

```
void generateData(SensorData &data, int count) {  
    std::random_device rd;  
    std::mt19937 gen(rd());  
    std::normal_distribution<float> temp_dist(25.0, 5.0);  
    std::uniform_int_distribution<int> time_dist(1, 5);  
  
    int timestamp = START_TIMESTAMP;  
    for (int i = 0; i < count; ++i) {  
        data.timestamps[i] = timestamp;  
        data.temperatures[i] = temp_dist(gen);  
        timestamp += time_dist(gen); // Increment timestamp  
    }  
}
```



SIMD Query



SIMD Query

Task: Calculate the average temperature within a timestamp range

Timestamps	1	2	5	7
Temperature	25.5	26.0	27.2	28.3

> 2	0	1	1	0
< 6	1	1	1	0
> 2 AND < 6	0	1	1	0

Masked Temps	0	26.0	27.2	0
Sum	53.2			

SIMD Query: Loading Data

Timestamps	1	2	5	7
Temperature	25.5	26.0	27.2	28.3

```
int32x4_t ts_vec = vld1q_s32(&data.timestamps[i]);    // Load 4 timestamps
```

```
float32x4_t temp_vec = vld1q_f32(&data.temperatures[i]); // Load 4  
temperatures
```

SIMD Query: Filtering Timestamps

> 2	0	1	1	0
< 6	1	1	1	0
> 2 AND < 6	0	1	1	0

```
uint32x4_t in_range_mask =  
    vandq_u32(vcgeq_s32(ts_vec, vdupq_n_s32(startTimestamp)),  
              vcleq_s32(ts_vec, vdupq_n_s32(endTimestamp)));
```


SIMD Query: Mask Application

Masked Temps	0	26.0	27.2	0
--------------	---	------	------	---

```
float32x4_t masked_temps = vmulq_f32(temp_vec,  
vcvtq_f32_u32(in_range_mask));
```

```
sum_vec = vaddq_f32(sum_vec, masked_temps);
```


SIMD Query: Final Steps

Sum	53.2			
-----	------	--	--	--

```
float total_sum = vaddvq_f32(sum_vec);  
for (int i = count - (count % 4); i < count; ++i) {  
    if (data.timestamps[i] >= startTimestamp &&  
        data.timestamps[i] <= endTimestamp) {  
        total_sum += data.temperatures[i];  
    }  
}
```

Advanced SIMD: Hash Join Algorithm

```
// Load probe keys
int32x4_t probe_keys = vld1q_s32(&probe_table.keys[i]);
// Compute hash
int32x4_t hash_vec =
    vmodq_s32(vmulq_s32(probe_keys, hash_multiplier),
hash_table_size);
// Gather hash table values
int32x4_t hash_table_vals = vld1q_s32(&hash_table[hash_vec]);
// Compare keys
uint32x4_t match_mask = vceqq_s32(probe_keys, hash_table_vals);
```


Advanced SIMD: Hash Join Algorithm

```
// Load probe keys
int32x4_t probe_keys = vld1q_s32(&probe_table.keys[i]);
// Compute hash
int32x4_t hash_vec =
    vmodq_s32(vmulq_s32(probe_keys, hash_multiplier),
hash_table_size);
// Gather hash table values
int32x4_t hash_table_vals = vld1q_s32(&hash_table[hash_vec]);
// Compare keys
uint32x4_t match_mask = vceqq_s32(probe_keys, hash_table_vals);
```

Advanced SIMD: Hash Join Algorithm

```
// Load probe keys
int32x4_t probe_keys = vld1q_s32(&probe_table.keys[i]);
// Compute hash
int32x4_t hash_vec =
    vmodq_s32(vmulq_s32(probe_keys, hash_multiplier),
hash_table_size);
// Gather hash table values
int32x4_t hash_table_vals = vld1q_s32(&hash_table[hash_vec]);
// Compare keys
uint32x4_t match_mask = vceqq_s32(probe_keys, hash_table_vals);
```


Advanced SIMD: Hash Join Algorithm

```
// Load probe keys
int32x4_t probe_keys = vld1q_s32(&probe_table.keys[i]);
// Compute hash
int32x4_t hash_vec =
    vmodq_s32(vmulq_s32(probe_keys, hash_multiplier),
hash_table_size);
// Gather hash table values
int32x4_t hash_table_vals = vld1q_s32(&hash_table[hash_vec]);
// Compare keys
uint32x4_t match_mask = vceqq_s32(probe_keys, hash_table_vals);
```

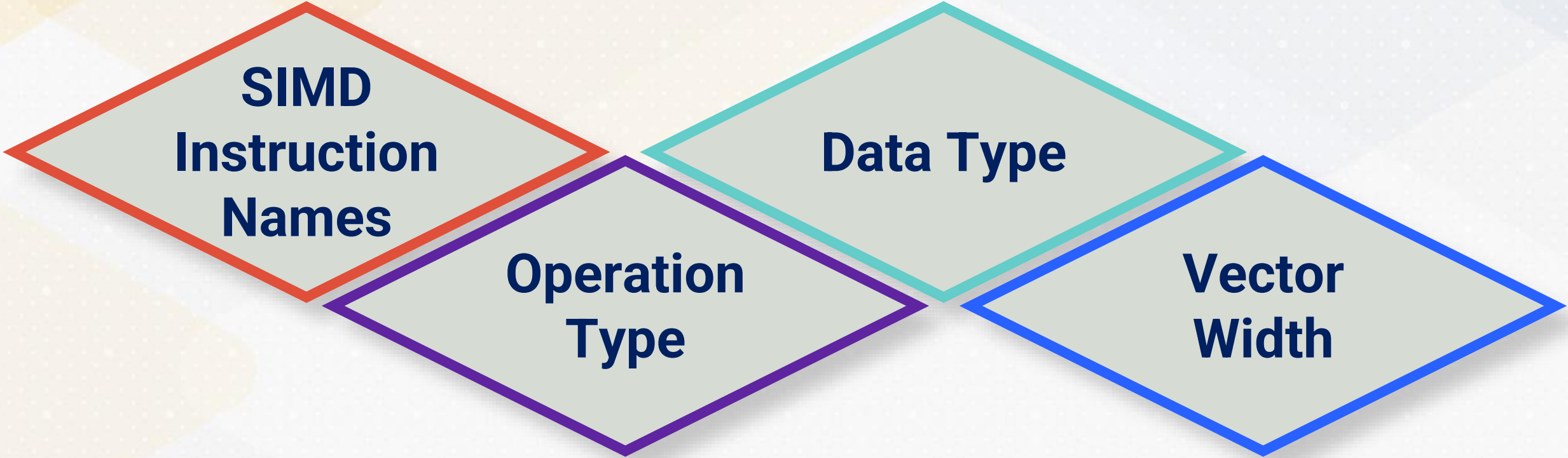
Advanced SIMD: Hash Join Algorithm

```
// Load probe keys
int32x4_t probe_keys = vld1q_s32(&probe_table.keys[i]);
// Compute hash
int32x4_t hash_vec =
    vmodq_s32(vmulq_s32(probe_keys, hash_multiplier),
hash_table_size);
// Gather hash table values
int32x4_t hash_table_vals = vld1q_s32(&hash_table[hash_vec]);
// Compare keys
uint32x4_t match_mask = vceqq_s32(probe_keys, hash_table_vals);
```


SIMD Instruction Naming

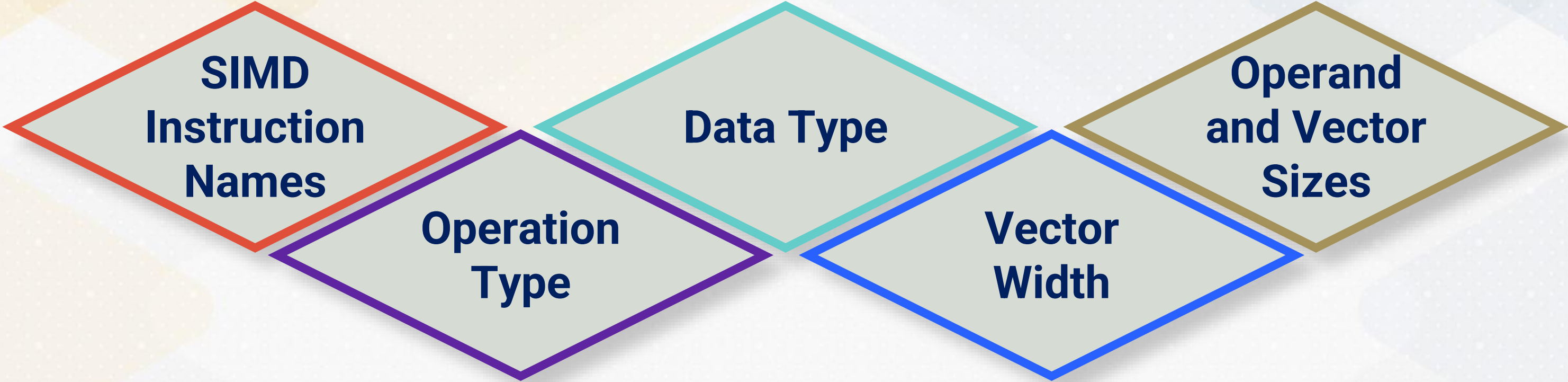
vaddq_f32	Addition Operation	32-bit floating-point numbers	Quad-word (4 floats)
vcgeq_s32	Compare greater than or equal to	Operates on signed 32-bit integers.	64-bit vector (2 integers)

SIMD Instruction Naming



vaddq_f32	Addition Operation	32-bit floating-point numbers	Quad-word (4 floats)
vcgeq_s32	Compare greater than or equal to	Operates on signed 32-bit integers.	64-bit vector (2 integers)

SIMD Instruction Naming



vaddq_f32	Addition Operation	32-bit floating-point numbers	Quad-word (4 floats)
vcgeq_s32	Compare greater than or equal to	Operates on signed 32-bit integers.	64-bit vector (2 integers)





Vector-at- a-Time Processing



**Vector-at-
a-Time
Processing**

**SIMD
Instructions**